

12e — Event Sourcing

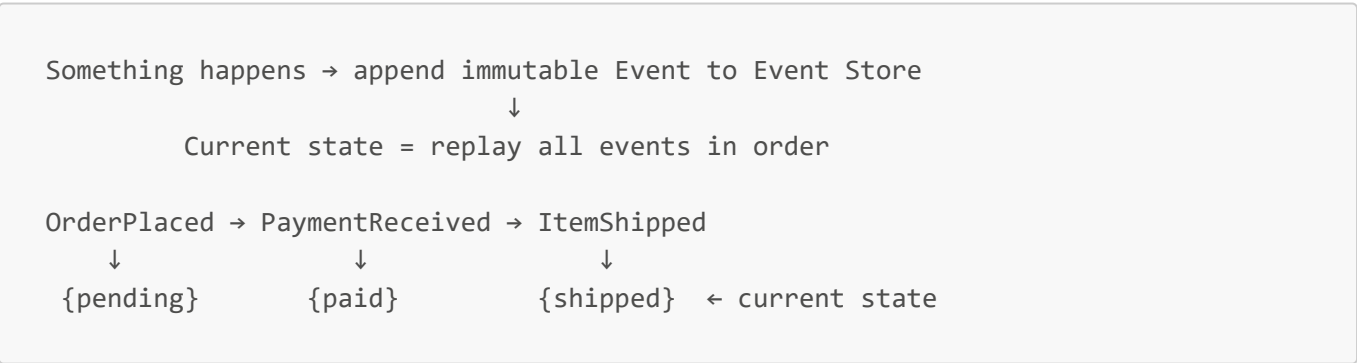
Key Concepts

- **Event Sourcing** = Store every event that led to the current state, instead of the state itself
- Current state is **derived** by replaying all past events in order
- Events are **immutable** — never updated, never deleted, only appended
- Event Store = append-only database that holds all events
- Often paired with CQRS

Traditional DB vs Event Sourcing

	Traditional DB	Event Sourcing
What's stored	Current state	All events that led to state
History	Lost on update	Full history always available
Audit log	Extra work to build	Free — it's the data model
Time travel	Not possible	Replay to any point in time
Complexity	Simple	Higher — need projections
Storage	Less	More (all events kept forever)
Debugging	Hard — state already changed	Replay exact sequence

How It Works



Three Rules

1. Something happens → record it as an immutable Event
2. Events are appended to Event Store — never updated or deleted
3. Current state = replay all events in order

Event Store

- Append-only database — optimized for sequential writes
- Events stored in order with timestamps
- Can replay from any point in time

Popular options: EventStoreDB, Apache Kafka, Azure Event Hub, PostgreSQL (append-only table)

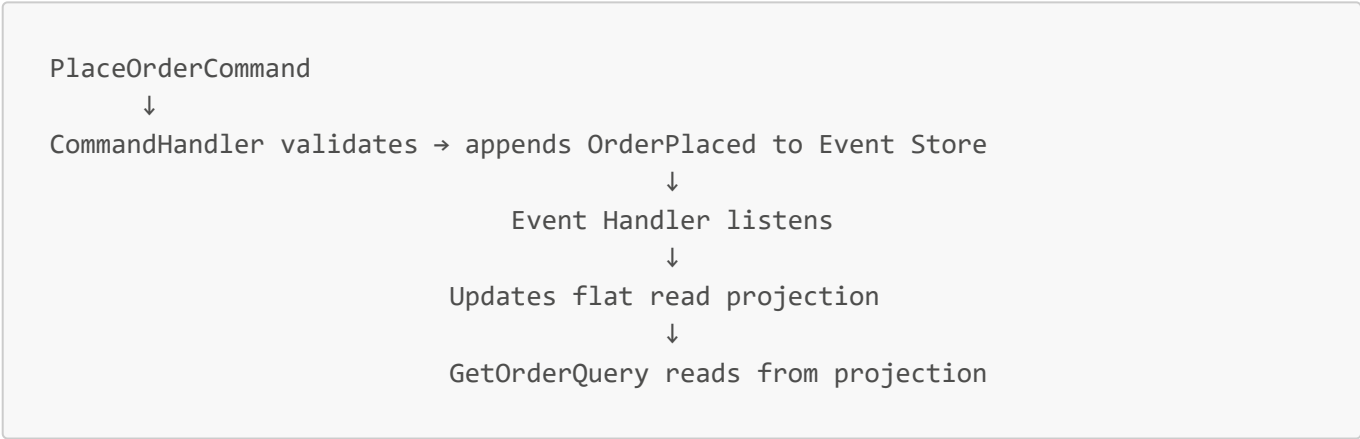
Snapshots — Performance Optimization

Replaying thousands of events on every read is slow. Snapshots fix this:

```
Every N events → save snapshot of current state
Next read → load latest snapshot + replay only events after it
```

Like a video game save point — don't restart from the beginning every time.

Event Sourcing + CQRS Together



- **Event Store** = source of truth (write side)
- **Projections** = optimized read models (query side)

Capabilities Unlocked

Capability	What it means
Audit trail	Full history of everything — free, no extra work
Time travel	Rebuild state at any point in the past
Replay	Re-process events through new business logic
Debug	Reproduce any bug by replaying exact event sequence
New projections	Build new read model by replaying all past events

Industry Examples

Company	Event Sourcing Application
Banking	Every transaction is an event — balance derived from transaction history
Uber	Trip state from events: TripRequested → DriverAssigned → TripStarted → TripCompleted
LinkedIn	Activity feed built by replaying user action events
Amazon	Order lifecycle as events — full audit trail for dispute resolution
Git	Every commit is an immutable event — code state derived by replaying all commits

When to Use Event Sourcing

Use when	Avoid when
Full audit trail required (banking, compliance)	Simple CRUD app — massive overkill
Time-travel debugging needed	Team unfamiliar with the pattern
Complex event-driven microservices	Read performance critical without projections set up
Domain has complex state transitions	

Interview Questions

Q1: What is Event Sourcing?

Store every event that led to the current state instead of the state itself. Current state is derived by replaying all past events in order.

Q2: What's the key difference from a traditional database?

Traditional DB stores current state — history is lost on update. Event Sourcing stores all events — nothing is ever overwritten or deleted.

Q3: What is an Event Store?

An append-only database — events written in order, never updated or deleted. Options: EventStoreDB, Kafka, Azure Event Hub.

Q4: What are Snapshots and why are they needed?

A snapshot saves current state every N events so you don't replay from the beginning every time. Load latest snapshot + replay only events after it.

Q5: Why are CQRS and Event Sourcing often used together?

Commands append events to the Event Store. Event handlers update read-side projections. Queries read from fast projections — clean separation of write and read concerns.

Q6: When would you NOT use Event Sourcing?

Simple CRUD apps — overkill. High complexity, more storage, projections need maintaining. Only justified when audit trail, time travel, or complex event-driven flows are required.